```
# Week 6: Propositional Logic
# Truth Table Enumeration Algorithm

# Knowledge Base
def KB(model):
    P = model['P']
    Q = model['Q']
    R = model['R']

    return ((not Q or P) and
            (not P or not Q) and
            (Q or R))


# Query 1: R
def query1(model):
    return model['R']


# Query 2: R -> P
def query2(model):
    return (not model['R']) or model['P']


# Query 3: Q -> R
def query3(model):
    return (not model['Q']) or model['R']
```

```
# Truth Table Entailment
def TT_ENTAILS(KB, alpha, symbols):
  return TT_CHECK_ALL(KB, alpha, symbols, {})



# Recursive checking
def TT_CHECK_ALL(KB, alpha, symbols, model):

  if len(symbols) == 0:

    if KB(model):
      return alpha(model)
    else:
      return True

  else:
    P = symbols[0]
    rest = symbols[1:]

    return (
      TT_CHECK_ALL(KB, alpha, rest,
            {**model, P: True})
      and
      TT_CHECK_ALL(KB, alpha, rest,
            {**model, P: False})
    )
```

```
# Main program
symbols = ['P', 'Q', 'R']

queries = [
    ("R", query1),
    ("R -> P", query2),
    ("Q -> R", query3)
]

# Display truth table
print("P Q R | KB")

from itertools import product

for values in product([False, True], repeat=3):
    model = dict(zip(symbols, values))

    print(
        model['P'],
        model['Q'],
        model['R'],
        "|",
        KB(model)
    )

# Check all queries
print("\nENTAILMENT RESULTS")
```

```
for name, alpha in queries:

    result = TT_ENTAILS(KB, alpha, symbols)

    print("KB entails", name, ":", result)
```

```
P Q R | KB
False False False | False
False False True | True
False True False | False
False True True | False
True False False | False
True False True | True
True True False | False
True True True | False

ENTAILMENT RESULTS
KB entails R : True
KB entails R -> P : False
KB entails Q -> R : True
```

```
]:
```